Hardware Design and Functional Verification Learning Roadmap Informational Document.

This document will serve as my pre-defined learning roadmap to guide me through my journey of learning, developing and verifying increasingly more complex concepts in order to enhance my verification knowledge overall.

1. Project purpose:

This roadmap has 3 purposes:

1. **Hardware Design**: Teaching myself digital hardware hands-on.
2. **Functional Verification**: Developing a library of components that will then be verified. The techniques will be selected according to the DUT's complexity and requirements.
3. **Computer Architecture**: Overall brush-up/teaching of individual, gradually more complex components that make up a larger system.

Every project should produce a list of things to implement and a list of results.

Therefore, for every component the development process will follow these steps:

1. Studying the component's architecture, purpose, etc.
2. Writing specification documents defining the intended functionality.
3. Designing and implementing the RTL code.
4. Preparing a verification plan based on the specifications.
5. Developing a verification environment.
6. Executing and analyzing simulations.
7. Evaluating functional coverage.
8. Applying formal verification where appropriate.
9. Introducing intentional design flaws for more complex projects to evaluate verification environment robustness.
10. Documenting the implementation, verification methodology, results and conclusions.



Required.



For more complex designs.

The roadmap is divided into 8 main study areas:

1. Sequential Logic and Storage
2. Clock Domains and Clock Domain Crossing (CDC)
3. Computational Datapaths
4. Control Logic and Arbitration
5. Communication Protocols and Bus interfaces
6. Memory Subsystems
7. Processor Architecture
8. Complete SoC Integration

2. General development methodology:

Step 1: Study the component.

Research the component before implementing to understand it's purpose, internal architecture, principles, etc.

Step 2: Write the specification document.

Should include:

- Component description and purpose.
- Functional requirements.
- Input and Output interfaces.
- Parameter definitions and constraints.
- Clock and reset requirements.
- Timing requirements.
- Operational behavior.
- Supported operating modes.
- Error conditions and unsupported operations.
- Expected behavior under simultaneous or conflicting operations.

Each requirement should have a unique identifier where practical for easier tracing.

Step 3: Design and implement the RTL model.

- Follow the specification.
- Be parametrized where appropriate.

- Use clear and consistent RTL coding and formatting.
- Be documented sufficiently to explain important decisions through similarly formatted comments.

Step 4: Write the verification plan.

- Functional requirements must be identified.
- Normal operating scenarios.
- Boundary conditions.
- Corner cases.
- Illegal or unexpected input conditions.
- Relevant verification techniques.
- Assertions and properties.
- Functional coverage points,
- Code coverage objectives.
- Formal verification opportunities.
- Expected verification completion criteria.

Step 5: Develop the verification environment.

Step 6: Execute verification.

Validate each verification concern with documented results.

Step 7: Evaluate coverage.

Step 8: Apply formal verification where appropriate.

Step 9: Introduce purposeful bugs.

Document the results in order to validate the robustness of the environment.

Step 10: Document the results.

- Design summary
- Verification environment architecture

- Verification techniques used
- Justifications
- Test scenarios executed (with examples)
- Coverage results
- Bugs discovered
- Limitations
- Conclusions and possible improvements

Continue with the following document to read about project planning: Hardware Design and Functional Verification Learning Roadmap - Project Plan